



FIAP - Faculdade de Informática e Administração Paulista

Domain Drive Design Using Java

Ana Carolina Pereira Fontes

Daniel Nicolas Leoterio

Matheus Moya de Oliveira

CHALLENGE PROJECT – SPRINT :

APLICAÇÃO DA TECNOLOGIA EM TELEMEDICINAS

**PARA A SOLUÇÃO DO ABSTENCIONISMO NO INSTITUO IREA-
HOSPITAL DAS CLINICAS**

SUMÁRIO

Integrantes.....	3
Objetivo e escopo	3
Funcionalidades	3
Protótipo do sistema	6
Modelo de entidade de relacionamento (MER)	7
Diagrama de classe	7
Procedimento para rodar a aplicação	9

Integrantes

- Ana Carolina Pereira Fontes, RM562145, 1TDSPV
- Daniel Nicolas Leoterio, RM562186, 1TDSPV
- Matheus Moya de Oliveira, RM562822, 1TDSPV

Objetivo e escopo

O sistema desenvolvido é um aplicativo de linha de comando para gestão básica de Pacientes, Médicos e Consultas. Ele permite cadastrar, consultar, atualizar e excluir registros (CRUD), além de agendar, reagendar, concluir e cancelar consultas, aplicando regras simples de negócio (como verificação de choque de agenda para o mesmo médico no mesmo horário). Junto de persistência em Oracle via JDBC, camadas de Model, DAO/Repository, Service (regras) e UI (menu console).

Funcionalidades

O projeto será desenvolvido em três módulos principais: Pacientes, Médicos e Consultas cada um com funcionalidades específicas que permitem cadastrar, alterar, buscar e excluir informações.

1. Pacientes

O módulo de pacientes tem como objetivo gerenciar todas as informações relacionadas aos usuários do sistema. As principais funcionalidades incluem:

- **Cadastro de pacientes** com os campos: nome completo, CPF, data de nascimento, telefone e e-mail.
- **Listagem completa** de todos os pacientes cadastrados.
- **Busca por CPF**, permitindo localizar rapidamente um paciente específico.
- **Atualização de dados** cadastrais, como nome, CPF, telefone e e-mail.
- **Exclusão de pacientes** a partir de seu ID.

Regras e validações:

- O **CPF deve ser único** no sistema. Caso já exista um registro com o mesmo CPF, o sistema bloqueia o cadastro e exibe a mensagem: *"CPF já cadastrado"*.
- Os dados são armazenados de forma padronizada:
 - CPF sem máscara no banco de dados e exibido formatado como 999.999.999-99.
 - Telefone exibido no formato (99) 99999-9999.
 - Datas no formato dd/MM/yyyy.

Camadas da aplicação:

- **DAO/Repository:** A classe `PacienteRepositoryJdbc` será responsável pelas operações de acesso ao banco de dados, implementando métodos como `save`, `update`, `delete`, `findById`, `findByCpf` e `findAll`.
- **Service:** A classe `PacienteService` centralizará as validações (por exemplo, checagem de CPF duplicado) e fará a ponte entre a camada de negócio e o repositório.

2. Médico

O módulo de médicos gerencia os profissionais de saúde cadastrados no sistema, com funcionalidades semelhantes às do módulo de pacientes:

- **Cadastro de médicos** com os campos: nome completo, CRM, especialidade, telefone e e-mail.
- **Listagem completa** de todos os médicos cadastrados.
- **Busca por CRM** para localizar profissionais específicos.
- **Atualização de dados** cadastrais.
- **Exclusão** de registros a partir do ID.

Regras e validações:

- O CRM deve ser **único**, impedindo cadastros duplicados.
- O telefone deve ser exibido no formato padronizado (99) 99999-9999.

Camadas da aplicação:

- **DAO/Repository:** A classe `MedicoRepositoryJdbc` será responsável pelas operações CRUD e pela busca por CRM.
- **Service:** A classe `MedicoService` centralizará as validações, como a verificação de CRM duplicado, e repassará os dados ao repositório.

3. Consultas

O módulo de consultas gerencia o agendamento e o controle das consultas médicas realizadas na plataforma. As principais funcionalidades incluem:

- Agendamento de consultas associando paciente, médico, data, hora e observações adicionais.
- Listagem completa de todas as consultas cadastradas.
- Cancelamento, conclusão e reagendamento de consultas, permitindo alterações na data e hora conforme necessidade.
- Exclusão de consultas do sistema.

4. Menu Principal

O sistema foi desenvolvido com base em um menu principal interativo no console, que apresenta as seguintes opções para o usuário:

- Pacientes
- Médicos
- Consultas
- Sair

Cada módulo possui um submenu próprio com as operações disponíveis, permitindo ao usuário interagir de forma simples e intuitiva com o sistema.

- Agendar nova consulta
- Cancelar consulta existente
- Concluir consulta realizada
- Reagendar consulta para nova data/hora
- Excluir consultas do sistema

Todos os campos de entrada são validados e as mensagens exibidas seguem prompts padronizados, garantindo uma experiência consistente. Após cada operação, o sistema pausa e solicita que o usuário pressione <Enter> para prosseguir, permitindo a leitura dos resultados apresentados.

Padrões Técnicos

Modelos (Entidades):

- Paciente
- Medico
- Consulta
- StatusConsulta

DAO/Repository: classes ...RepositoryJdbc usando JDBC e PreparedStatement.

Service: regras e validações (ex.: unique keys, conflito de agenda).

Utilitários: DbConnection (Oracle via db.properties), ConsoleIO (leitura/pausa/máscaras), Formatters (CPF, telefone, data e datetime).

Testes manuais: via menu de console.

Teste automático rápido: classe CrudSmokeTest exercitando Create/Read/Update/Delete e regras críticas (ex.: conflito de agenda).

Protótipo do sistema



Modelo de entidade de relacionamento (MER)

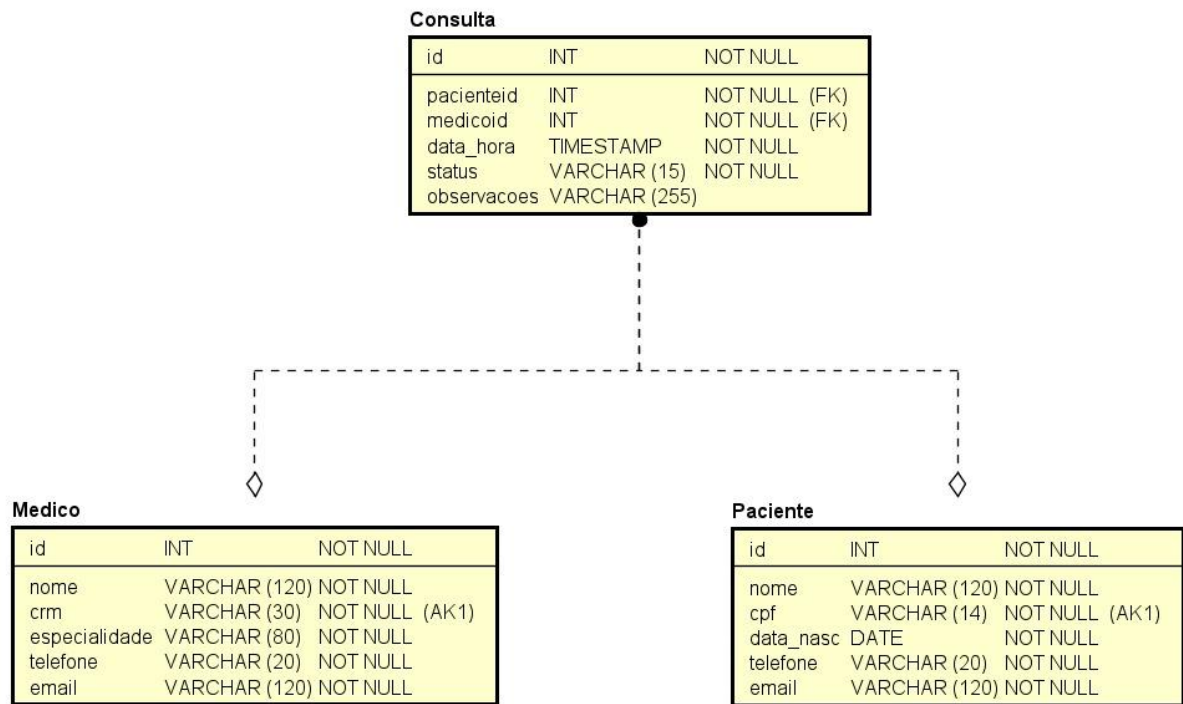
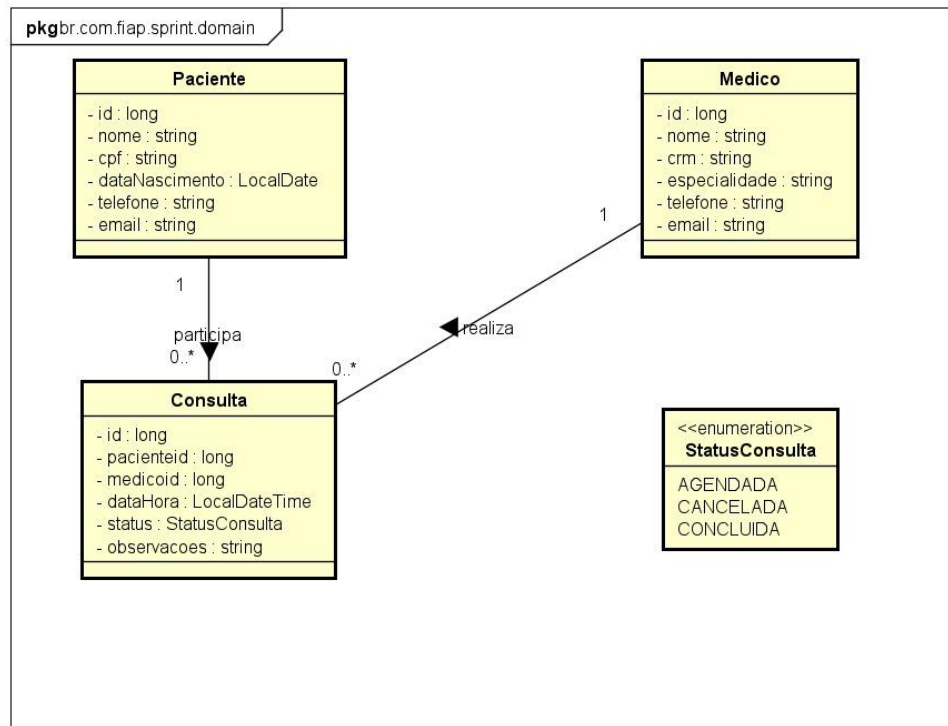
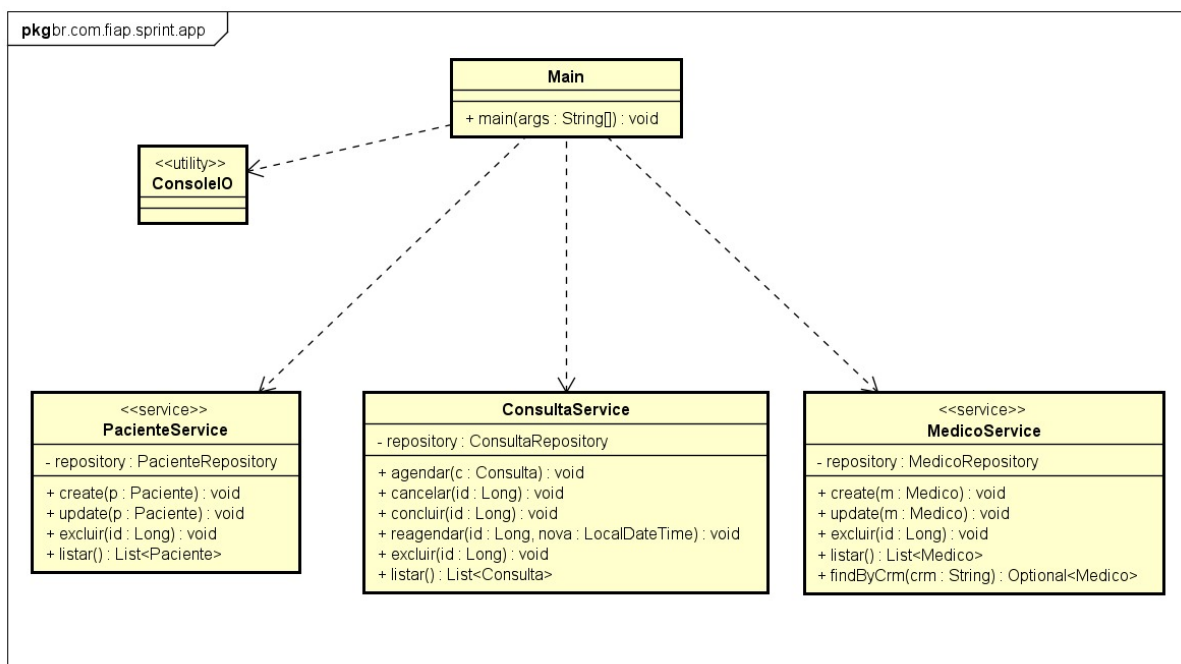
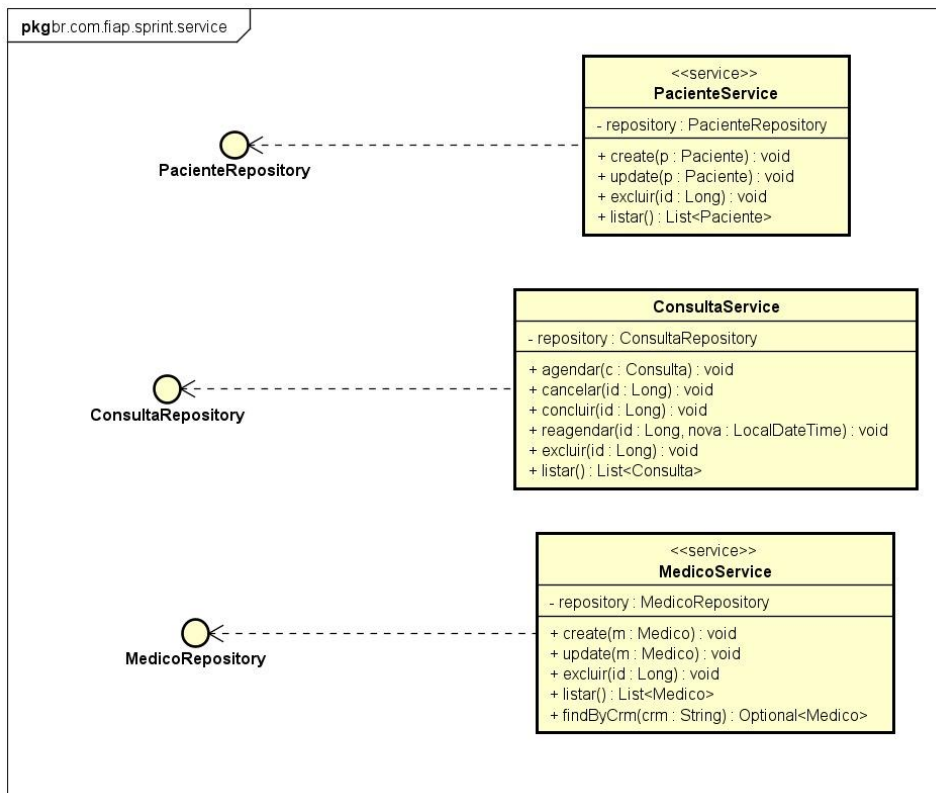
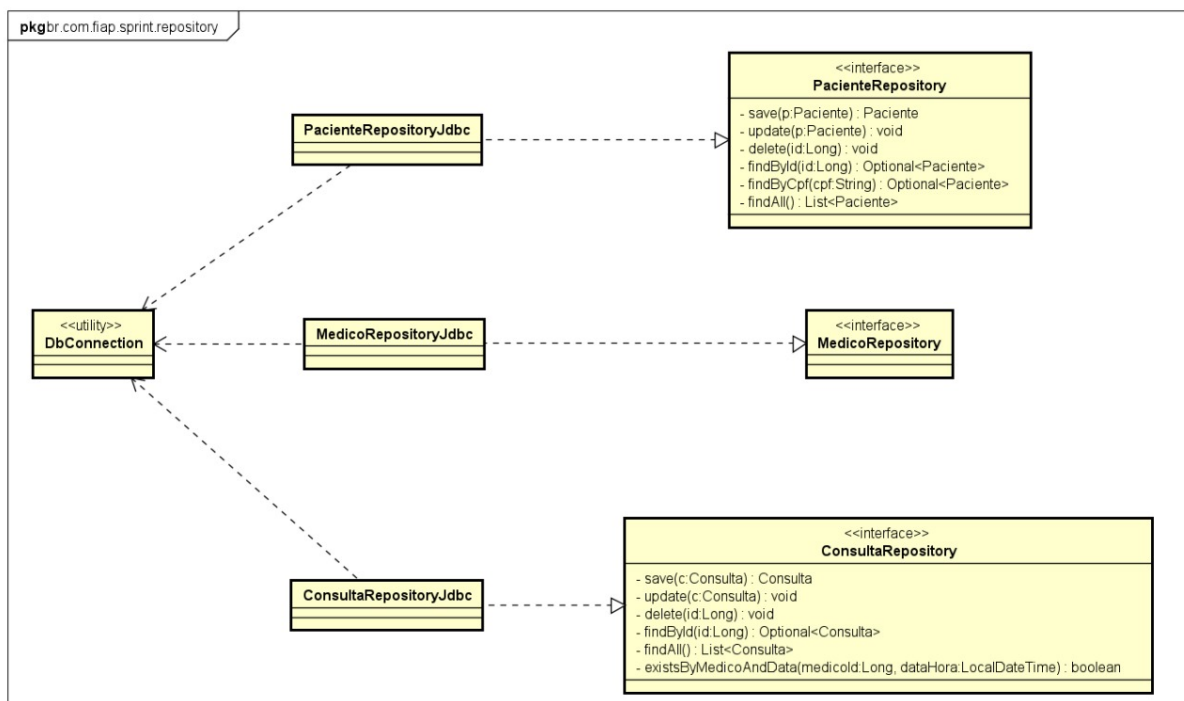
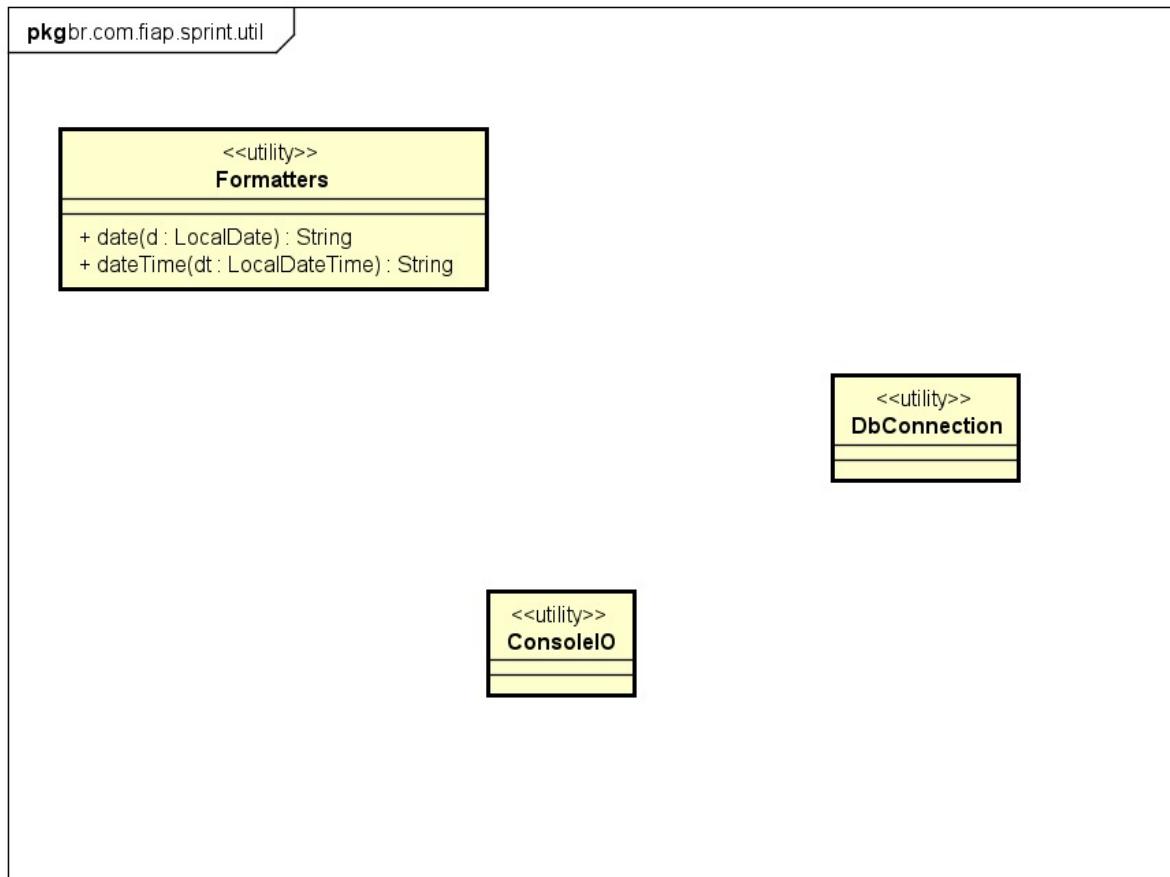


Diagrama de classe







Procedimento para rodar a aplicação

São Paulo
2025

- Ferramentas utilizadas: IntelliJ
- Versões necessárias: Java JDK, Maven, Oracle Database
- Tratamento de console:

CPF: Normalizado para ###.###.###-##

Telefone: Normalizado para (##) 9#####-####

Data de nascimento: dd/mm/yyyy

Data/hora de consulta: dd/mm/yyyy hh:mm

Cadastre ao menos 1 paciente e 1 médico antes de agendar consultas